

A Brave New Compiler

Author: Ludwig Alvarado

It is the year 632 AC (After ChatGPT, 2654 in the Gregorian calendar). People around the world have forgotten how to deal with everyday problems, living in a permanent state of pleasure and automation. Even the programming world is slowly disappearing, as most software is now built with Electron-based JavaScript/TypeScript and C# applications, and nearly all programs are generated by machines. Humans no longer need to think critically. Because of this, most code in the world has become bloated, and even simple programs feel slow and inefficient, from airplane schedulers to operating systems.

The Liberation Initiative for Non-bloated Unified eXecution (LINUX) is one of the few remaining groups that still maintains the GNU C Compiler (GCC). Most JavaScript/TypeScript frameworks depend on it, but only a few programmers are still able to maintain this critical codebase, and they are growing old. Yerman Huxley is a young man from *Ciudad Kennedy Altman*. He dislikes modern JS/TS and Electron-based C# applications. Instead, he is deeply committed to the ideals of LINUX...he likes writing compilers. If he studies with enough dedication, a revolution might still be possible.

The source code of GCC has been lost, and Yerman cannot compile programs anymore, so he is writing a new compiler from scratch. This new C compiler is called: "Compiler for Anti-Bloat and Independent Toolchain Operations" (CABITO). Yerman has made significant progress, but he is currently struggling with the bracket matching system. It is well known that a source code is syntactically valid if all brackets are properly matched: an opening bracket (must be closed by), and similarly for [with] and { with }. However, different types of brackets must not be mixed.

A simple C program has this form:

```
int main(){
    printf("Hello World\n");
    return 0;
}
```

Extracting all brackets, we obtain a string of the form:

(){}()

Examples of valid bracket sequences are:

- (((()))
- {((({[[[({})]]]))})}
- {[()]}

An invalid C program may look like this:

```
int main[]{
    printf("Hello World\n");
    return 0;
}
```

Extracting all brackets, we obtain:

[]{}()

Here, [and) do not match, and { is also incorrectly paired with).

Another example of invalid brackets is:

- {}

Input

You will be given a string s ($1 \leq |s| \leq 10^7$) that contains only six possible characters: [,], (,), {, }. The input ends when the string 0 is read, and this case should not be processed.

Output

For each test case, print "YES" if all brackets in the string are correctly matched, and "NO" otherwise.

Sample test cases

Sample case #1	
Input	Output
((()))	YES
{(({{[[[({})]]]})})}	YES
[]{}()	NO
()	NO
0	

Notes

In the first test case, all brackets are correctly opened and closed in the proper order.

In the third case, there is a mismatch between different types of brackets, making the sequence invalid.

Important Aspects

The following points describe how this exam problem will be evaluated and what restrictions must be considered during the solution process:

1. The test will be solved in **two stages**:

- Design and explanation of the algorithm on paper (**3.0 points**).
- Correct implementation in the code editor (**2.0 points**).

Both stages are graded independently. A correct idea with poor implementation, or correct code with no reasoning, may lose points.

2. This exercise evaluates topics studied up to **stacks and queues**.
3. No external libraries are required for this problem. If you have questions ask professor or monitor.
4. Do not validate any input data, they are correctly provided
5. Output format must match exactly the judge expectation.

Any missing links, extra spaces, duplicated nodes, infinite loops, or incorrect termination will be judged as wrong answer.